This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

Cleaning Uncertain Data with Crowdsourcing - a General Model with Diverse Accuracy Rates

Chen Zhang, *Member, IEEE,* Weiteng Xie, Nan Liu, Qifan Li, Kaishun Wu, *Member, IEEE,* Di Jiang, Peiguang Lin, Lei Chen, *Member, IEEE,* and Haodi Zhang

Abstract—Uncertain data has been emerged as an important problem in database systems due to the imprecise nature of many applications. To handle the uncertainty, probabilistic databases can be used to store uncertain data, and querying facilities are provided to yield answers with confidence. However, the uncertainty may propagate, hence the results from a query or mining process may not be useful. In this paper, we leverage the power of crowdsourcing by designing a set of Human Intelligence Tasks (HITs) to ask a crowd with diverse accuracy rates, to improve the quality of uncertain data. Each HIT is associated with a cost, thus, we need to design solutions to maximize the data quality with minimal number of HITs. There are two obstacles for this non-trivial optimization, which lead to very high computational cost for selecting the optimal set of HITs. First, members of a crowd may return incorrect answers with different probabilities. Second, the HITs decomposed from uncertain data are often correlated. We have addressed these challenges in this paper by designing an effective approximation algorithm and an efficient heuristic solution, even under diverse individual accuracy rates of the crowdsourcing workers. To further improve the efficiency, we derive tight lower and upper bounds for effective filtering and estimation. The solutions are verified with extensive experiments on both a simulated crowd and a real crowdsourcing platform.

Index Terms—crowdsourcing, cleaning uncertain data, approximation algorithm, heuristic algorithm.

1 Introduction

Uncertain data has been emerged as an important problem in database systems due to the imprecise nature of many applications. In the field of information extraction, some learning-based model (e.g. Conditional Random Field) usually produces a ranked list of extractions, each of which is associated with a probability of correctness [1]. In entity resolution and schema matching, automatic tools may generate multiple results, each of which is associated with a confidence value indicating the matching quality [2], [3]. When data from multiple data sources are integrated, conflicting information may be handled by keeping conflicted data associated with probabilities, a.k.a uncertain/probabilistic data.

Due to the existence of an exponential number of possible worlds in an uncertain database (a possible world is one possible combination of real instances of uncertain data), mining uncertain data is often computationally expensive. Moreover, the uncertainty may propagate, so the returned results from a query may not be useful. Thus, it is desirable to improve the data quality by reducing the uncertainty. In many applications, data becomes uncertain because computers have difficulties to recognize human-intuitive semantics via the given data representation (e.g. images, natural language) or handle human-intrinsic tasks (e.g. extracting the correct information or language translation). Therefore, intuitively, human perception should be very effective in cleaning uncertain data.

Existing works of uncertain data cleaning [4], [5] assume there is a cleaning agent (e.g. an expert) available, which may access and return the 'ground truth' of the uncertain data. However, in many circumstances, such availability does not hold. Along with the emerging of uncertain data, crowdsourcing has become a hot research topic due to the success of several crowdsourcing platforms such as Amazon Mechanical Turk, CrowdFlower and CrowdOTA [6]. These platforms provide human computational power, which is not only always available, but is at least fairly affordable. In this paper, we leverage the power of crowdsourcing for cleaning uncertain data. Compared with an earlier version of our paper [7], we adopt a more realistic setting and generalize the model.

1.1 Uncertainty Model and Quality Measurement

Among various models of uncertainty, we consider the *x-relation*, which is widely adopted in the field of uncertain database [4], [5], [8], [9], [10]. In the x-relation model, an x-relation contains a set of independent x-tuples. Each x-tuple includes a set of mutually exclusive tuples (or called alternatives), associated with probabilities, which represent a discrete probability distribution of these tuples being correct.

Table 1 illustrates a probabilistic database, where the uncertainty of each entry is captured by an x-tuple. In

- *C. Zhang and L. Chen are with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong, SAR China. Email: {czhangad, leichen}@cse.ust.hk.*
- *W. Xie, Q. Li, N. Liu and K. Wu are with the College of Computer Science and Software Engineering, Shenzhen University and Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen University, China. E-mail: {wtxie, qfli, nliu, wu}@szu.edu.cn.*
- *D. Jiang is currently a senior scientist at WeBank AI. E-mail: dijiang@webank.com*
- *P. Lin is with School of Computer Science and Technology, Shandong University of Finance and Economics, Jinan China. Email: linpg@sdufe.edu.cn.*
- *H. Zhang, the corresponding author, is with the College of Computer Science and Software Engineering, Shenzhen University and Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen University, China. E-mail: hdzhang@szu.edu.cn*

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

TABLE 1
Location address of T_1: 51A Hayward East New York

Xid	Tid	House no	Area	City	$Pr(t_i)$
T_1	t_1	51	Hayward	New York	0.4
T_1	t_2	51A	Hayward East	New York	0.3
T_1	t_3	51A	Hayward East	York	0.2
T_1	t_4	51	East	York	0.1

TABLE 2
Candidate HITs for Cleaning with Crowd Workers, and Distributions of the Ground Truth and Crowdsourced Answers

ID	HIT content	$Pr(h_c)$	P_{cr}	$Pr(A_c)$
HIT1	Is the House no 51?	y(0.5) n(0.5)	0.90	y(0.5) n(0.5)
HIT2	Is the House no 51A?	y(0.5) n(0.5)	0.65	y(0.5) n(0.5)
HIT3	Is the Area Hayward?	y(0.4) n(0.6)	0.55	y(0.49) n(0.51)
HIT4	Is the Area Hayward East?	y(0.5) n(0.5)	0.70	y(0.5) n(0.5)
HIT5	Is the Area East?	y(0.1) n(0.9)	0.85	y(0.22) n(0.78)
HIT6	Is the City New York?	y(0.7) n(0.3)	0.60	y(0.54) n(0.46)
HIT7	Is the City York?	y(0.3) n(0.7)	0.75	y(0.4) n(0.6)

particular, each x-tuple represents an address of a facility location, provided by an information extraction model (e.g. CRF or HMM, and the creation of such probabilistic database from the extraction models is detailed in [1]). For instance, the x-tuple T_1 indicates four possible addresses for a location, i.e. $\{t_1,t_2,t_3,t_4\}$, for textual data "51A Hayward East New York"; and the probability of t_1 being correct is 0.4. Such a probabilistic data model well captures the correlation among the attributes (labels). In the example of Table 1, the labels 'House no', 'Area' and 'City' are correlated: the marginal probability $Pr(House\ no = 51A) = Pr(t_2) + Pr(t_3) = 0.5$, while conditional probability $Pr(House\ no = 51|Area = East) = 1$ and $Pr(House\ no = 51|Area = Hayward\ East) = 0$.

To measure the data quality of an x-tuple, we follow the proposal in [4], [5], [11] and use the *negative value of the Shannon entropy*, i.e. $\sum_p p \log p$. For instance, the quality of T_1 in Table 1, denoted by $Q(T_1)$, is computed as $Q(T_1) = 0.4*\log 0.4+0.3*\log 0.3+0.2*\log 0.2+0.1*\log 0.1 = -0.55$. This measure quantifies the ambiguity of an x-tuple: the more uncertain the data, the lower the quality. Our core objective is to improve the quality of x-tuples with the help of crowdsourcing.

1.2 Cleaning with Crowdsourcing Worker

In Table 1, we demonstrate an example of uncertain data, in which the uncertainty is essential because data extraction techniques cannot entirely 'understand' the location address written in English - a natural language. Therefore, it becomes critical to investigate the following question: where can we find the human perceptions concerning the given uncertain data? Fortunately, we have crowdsourcing as a promising solution. With the recent development of crowdsourcing in both academic and industrial communities, there are sophisticated on-line crowdsourcing platforms (e.g. Mechanical Turk, CrowdFlower) serving affordable sources of human perceptions. The expected information concerning the given uncertain data can be queried via publishing questions, named Human Intelligent Tasks (a.k.a. HITs), each of which would generate a monetary cost. Additionally, the work-flow of publishing HITs can be automated with available APIs (e.g. REST APIs). Facilitated with these, we can significantly improve the data quality by automatically issuing a number of HITs.

It is well-known that crowdsourcing works best when tasks can be broken down into very simple pieces [12], [13], [14]. An entire tuple (containing values of many attributes) may be too large a grain for crowdsourcing workers - each individual worker may have small quibbles with a proposed tuple, so that a simple question on the correctness of tuples may get mostly negative answers, with each user declaring it less than perfect. In other words, determining the correctness of an entire tuple is too difficult for crowdsourcing worker. On the other hand, asking open-ended questions is not recommended for crowdsourcing workers, because it may be difficult to integrate multiple suggestions of heterogeneous semantics. As a result, we propose to ask the crowdsourcing worker questions regarding individual cells. Each HIT is a question of form "Is the *c.att c*?" , where *c* and *c.att* represent a cell value and its corresponding attribute, respectively. Given an example in Table 1, there are seven candidate HITs available to be selected for crowdsourcing, as shown in Table 2. This kind of much simpler questions, in most circumstances, can be easily answered with a yes or no.

With the running example in Tables 1 and 2, we demonstrate how the crowdsourcing worker is going to clean the uncertain data by answering HITs, and improve the data quality as a consequence. Assuming that "I: City = New York" (i.e. HIT6) is confirmed to be true by a crowdsourcing worker. Then we have

$$\begin{aligned} Pr(t_1|I) &= \frac{Pr(t_1) \cdot Pr(I|t_1)}{Pr(I) = Pr(t_1) + Pr(t_2)} \\ &= \frac{0.4 \times 1}{0.4 + 0.3} = 0.57 > 0.5 \end{aligned} \tag{1}$$

Similarly $Pr(t_2|I) = 0.43, Pr(t_3|I) = 0$ and $Pr(t_4|I) = 0$. As a result, the quality $Q(T_1)$ is improved from -0.55 to -0.3.

In the above example, we assume that obtaining information I from crowdsourcing worker with 100% confidence, which is unrealistic if we consider the error rate of crowdsourcing worker. As pointed out in paper [15], even with the simple tasks such as labeling, the quality of normal worker (not sloppy worker or spammer) is around 75%. Clearly, this quality varies depending on many factors, including the format and domain of the HIT, the time of publication, the HIT interface etc. Sometimes the quality may be even worse than 75%. Nevertheless, such noisy answers may still be conductive as far as the uncertainty is well managed. Continued with the running example, we make a more realistic assumption: "I: City = New York" is answered by a crowdsourcing worker cr with general confidence 60%. Assuming the HIT is answered independently by a worker, we have

$$\begin{aligned} &Pr(t_1|e : I\ is\ obtained\ from\ a\ crowdsourcing\ worker) \\ &= Pr(t_1) \cdot Pr(e|t_1)/Pr(e) \\ &= \frac{Pr(t_1) \cdot Pr(cr\ is\ correct)}{Pr(I) \cdot Pr(cr\ is\ correct) + (1 - Pr(I)) \cdot Pr(cr\ is\ incorrect)} \\ &= \frac{0.4 \times 0.6}{0.7 \times 0.6 + 0.3 \times 0.4} = 0.44 \end{aligned}$$

Similarly, $Pr(t_2|e) = 0.34$, $Pr(t_3|e) = 0.15$ and $Pr(t_4|e) = 0.07$. The quality $Q(T_1)$ is hereby improved to -0.52. From

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

the above results, we can see that, even when the crowdsourcing worker is imperfect, the quality of uncertain data is still improved after cleaning. After crowdsourcing a few more HITs, the confidence of the correct tuple is increased to be close to 1, while the others approach to 0.

1.3 Challenges and Contributions

The above analysis shows that crowdsourcing provides a new opportunity for uncertain data cleaning. However, since each HIT is associated with a cost, we need to design solutions to maximize the data quality by selecting an optimal minimum set of HITs. This selection is not trivial due to the following two obstacles: first, crowdsourcing workers may return the incorrect answers with different probabilities; second, the HITs decomposed from an x-tuple are naturally correlated. These two obstacles lead to the NP-hardness for selecting the optimal set of HITs. To address this challenge, firstly, we design an approximation algorithm with approximation guarantee $(1+1/e)$; and secondly, we propose a heuristic solution, which is much more efficient than the approximation one, but has comparable effectiveness.

In our previous conference paper [7], we addressed this problem and assumed that each crowdsourcing worker committed to the same accuracy rate. While in this paper, we consider a more general and realistic situation: (1) Each HIT has a probability that it will be answered correctly according to the hardness of HIT; (2) Every crowdsourcing worker has a probability of answering HIT correctly, which indicates the credibility of the worker. Therefore, the model in [7] can be regarded as a special case (crowdsourcing workers answer HIT with the same accuracy). Under the two conditions above, we can compute the probability that HIT is answered correctly, and publish the k HITs selected by the model without assuming a unified accuracy of crowdsourcing workers.

We summarize our contributions as follows:

First, we indicate how to utilize noisy crowdsourced answers to improve the data quality of an x-tuple, and analyze the functional relations among the accuracy rates of crowdsourcing workers, data quality and a candidate HIT.

Second, we address the core optimization problem, namely, how to select the most profitable questions with a cardinality constraint. We prove that the expected quality improvement by asking these HITs is equivalent to the difference between the 'joint entropy' and 'sum of entropy values of crowdsourcing workers'. This greatly reduces the time complexity of our algorithm. We also prove that the finding the exact solution is NP-hard, and an approximation algorithm as well as a heuristic solution are proposed.

Third, compared with the earlier version of the paper [7], we remove the strong assumption that all crowdsourcing workers have the same accuracy rate. We prove that even with diverse accuracy rates, our algorithm is still optimal. Moreover, we derive tighter lower and upper bounds for effective filtering and estimation.

Last, we have conducted extensive experiments on both synthetic and real data sets. The experimental results show that the data quality is significantly improved via the power of crowdsourcing, even without the original assumption of uniform accuracy rate. We also compare our method with automatic cleaning tools such as HoloClean, and propose a natural way to combining the two approaches.

TABLE 3
MEANINGS OF SYMBOLS

T	an x-tuple
$t_1, \ldots, t_{\|T\|}$	tuples of T
$Pr(t_1), \ldots, Pr(t_{\|T\|})$	probabilities of tuples
$Q(T)$	data quality of T
$ATT = \{att_1, \ldots, att_{\|ATT\|}\}$	semantic attributes of T
c_{ij} (c for simplicity)	value of the cell intersecting t_i and att_j
$h_c/h_{c_{ij}}$	the cleaning HIT verifying c/c_{ij}
$Pr(h_c)$	probability of c being the correct value of att_j, i.e. the probability that the ground truth of $h_{c_{ij}}$ is 'yes'
m	the total number of candidate HITs
A_h	the crowdsourced answer of HIT h
P_{cr}	the accuracy of the crowd worker cr
$Pr(A_h)$	probability of crowdsourcing workers answering 'yes' to HIT h
$H(cr)$	the entropy of the crowdsourcing worker
$H(.)$	the entropy of a random variable
$t \models c$	t cell c contains the correct value if t is correct

2 Problem Formulation

In this section, we first formally introduce x-tuple together with its quality metric. Then, we define crowd-based cleaning operations, and finally formulate our problem of utilizing crowdsourcing to clean x-tuples.

2.1 Uncertain Data and Quality Metric

Definition 1 (x-tuple with semantic attributes). For a given table under schema Σ, which consists of a set of **semantic** attributes $ATT = \{att_1, att_2, ..., att_{|ATT|}\}$. Each tuple t_i consists of three components: a unique identifier id, a confidence $P(t_i)$ that is the probability of t_i being correct, and the set of semantic attributes ATT. An x-tuple T consists of one or more tuples i.e. $T = \{t_1, t_2, ..., t_{|T|}\}$. $|ATT|$ and $|T|$ are the size of ATT and T, respectively. Each of the tuples is associated with a probability of being correct. Let $Pr(t_i)$ denote the probability of t_i being correct, then

$$\sum_{i=1}^{|T|} Pr(t_i) \leq 1.$$

Please note that the *semantic* attributes indicate the attributes describing human-intrinsic information of an entity. Besides the semantic attributes, schema Σ may also consist of other attributes (e.g. IDs). In the example of Table 1, "House no", "Area" and "City" are semantic attributes, while others are not. Intuitively, only data under semantic attributes worth being crowdsourced.

Definition 2 (Data Quality [4], [5]). Given an x-tuple T, the quality of T, denoted by $Q(T)$, is the negative value of the Shannon entropy, i.e.

$$Q(T) = \sum_{i=1}^{|T|} Pr(t_i) \cdot \log Pr(t_i) \tag{2}$$

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

where $\sum_{i=1}^{|T|} Pr(t_i) = 1$. If $\sum_{i=1}^{|T|} Pr(t_i) < 1$, we conceptually insert into T a tuple t_{null}, with null values for all attributes and probability $Pr(t_{null}) = 1 - \sum_{i=1}^{|T|} Pr(t_i)$.

We are using the same mathematical metric as proposed in [4], [5], which is originally called *PWS-quality*. Shannon entropy quantifies the randomness of a random variable, and low randomness indicates high quality of uncertain data. However, we adopt the metric to evaluate the quality of an individual x-tuple, rather than the answer of a query. By improving the quality of x-tuples (i.e. the data), the quality of any query answers would be naturally improved.

2.2 Crowdsourcing Model

Definition 3 (cleaning HIT). Given x-tuple T, let c_{ij} be the cell intersecting tuple $t_i \in T$ and attribute att_j. A cleaning HIT (or HIT in short) corresponding to c_{ij}, denoted $h_{c_{ij}}$, is a human intelligent task asking the crowdsourcing worker to verify whether c_{ij} is the correct value for att_j in T. Moreover, let $Pr(h_{c_{ij}})$ be the probability that the correct answer of $h_{c_{ij}}$ is 'yes', then

$$Pr(h_{c_{ij}}) = \sum_{t_x \models c_{ij}} Pr(t_x) \tag{3}$$

where $t_x \models c_{ij}$ denotes c_{ij} is the value of att_j in tuple t_x, i.e. t_x entails c_{ij}. In other words, $t_x \models c_{ij}$ means that if t_x is correct, then c_{ij} is correct.

From Definition 3, we can see that the candidate HITs can be easily constructed for a given x-tuple T. Since different tuples may have the same value for an attribute, the probability of c_{ij} being the correct value of att_j (i.e. the correct answer of $h_{c_{ij}}$ is yes) can be computed by summing up the probabilities of tuples in which c_{ij} is the value of att_j. The total number of HITs constructed for t, denoted by m, is upper bounded by $|ATT| \cdot |T|$, i.e. the number of attributes multiply the number of tuples. To be more specific, $m = \sum_{i=1}^{|ATT|} opts(att_i)$, where $opts(att_i)$ is the number of all options for attribute att_i. Each HIT can be considered as a random variable following a Bernoulli distribution. The distributions of HITs in Table 1 are demonstrated. Please note that the HITs derived from the same x-tuple are correlated in general. In the example of Table 1, given that "the House no is 51A" (HIT2), we can easily infer that "the Area is Hayward East"(HIT4), and the probability of "the City is York"(HIT7) becomes 0.4 (similar to Equation 1).

Due to differences in skill levels, or the amount of time and effort spent, the answers returned by crowdsourcing workers may be imperfect. We model the potential noisy answers by a general error model for crowdsourcing workers, illustrated as follows.

Definition 4 (Crowdsourcing Worker's Accuracy). Given a crowd worker cr, cr's accuracy, denoted by $P_{cr} \in [0.5, 1]$, is the probability that cr correctly answers each HIT. In addition, each HIT is assumed to be answered independently. How to model the crowdsourcing workers is application-specific. While some works assume that the crowdsourced answers are 100% accurate [16], [17] [13], [18], we adopt a more general error model, which ensures that the answer returned by the crowdsourcing worker is always correct with a probability no lower than $1/2$,and each crowdsourcing worker with different accuracy rates,as shown in Definition 4. This is a classical crowdsourcing model widely used by a stream of works [19], [20], [21], [22], [23]. A crowdsourcing worker may have different accuracy for different domains. The accuracy for a domain can be easily estimated with a set of sample HITs with ground truth.

Definition 5 (Entropy of Crowdsourcing Worker). Given a crowdsourcing worker cr and its accuracy P_{cr}, the entropy of cr is

$$H(cr) = -P_{cr} \cdot \log P_{cr} - (1 - P_{cr}) \cdot \log (1 - P_{cr}) \tag{4}$$

Given the crowdsourcing worker's accuracy, $H(cr)$ is a **positive** constant measuring the randomness of the crowd worker's behavior.

Definition 6 (Entropy of Answer). Given an x-tuple T, a crowdsourcing worker with accuracy $P_{cr} \in [0.5, 1]$. Let A_c be the answer of h_c provided by the crowdsourcing worker, and $Pr(A_c)$ and $1 - Pr(A_c)$ be the probabilities of 'yes' and 'no' being answered, respectively. the entropy of A_c is

$$\begin{aligned} H(A_c) \quad = \quad & -Pr(A_c = yes) \cdot \log Pr(A_c = yes) \\ & -Pr(A_c = no)) \cdot \log Pr(A_c = no) \end{aligned}$$

To be convenient, in the rest of this paper, we regard h_c and A_c that appear in probability functions to be Boolean variables

h_c : the ground truth of HIT h_c is yes;

A_c : HIT h_c is answered yes by the crowdsourcing worker.

Thus, the entropy of A_c is

$$\begin{aligned} H(A_c) \quad = \quad & -Pr(A_c) \cdot \log Pr(A_c) \\ & -(1 - Pr(A_c)) \cdot \log (1 - Pr(A_c)) \end{aligned} \tag{5}$$

2.3 Problem Statement

Now, we formally define the problem of data cleaning via crowdsourcing as follows.

Definition 7 (Problem Statement). Given an x-tuple T, the budget B, crowdsourcing workers $C = \{cr_1, cr_2, \ldots, cr_k\}$, each with individual accuracy P_{cr_1},P_{cr_2},...,P_{cr_k}, our goal is to maximize the quality $Q(T)$ by selecting HITs and receiving at most B crowdsourced answers.

From the above statement, the crowd is considered as a tool for cleaning uncertain data. However, since crowd may be noisy and unreliable, the first concern is *how to use the noisy crowdsourced answers to improve data quality*. In Section 3, we present a series of theoretical analyses, proving that the expected improvement of data quality is always non-negative. Then, in Section 4, we optimize the crowdsourcing process by selecting the most profitable set of HITs.

3 Utilization of Crowdsourced Answers

A crowd can be noisy, hence each crowdsourced answer is inherently uncertain. In this section, we discuss how to use uncertain crowdsourced answers to improve the data quality of x-tuples.

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

3.1 Merging Crowdsourced Answers into X-tuples

Since both crowdsourced answers and x-tuple are essentially uncertain, the effect of crowdsourced answers on data quality can be treated as posterior probabilities conditioning on answers, which is properly addressed with Bayes' theorem.

First, for a given x-tuple T, we clarify the functional relations among the accuracy rates of crowdsourcing workers $P_{cr_1}, P_{cr_2}, ..., P_{cr_k}$, data quality $Q(T)$ and a candidate HIT h_c. As mentioned above, please note the difference between h_c and A_c: h_c denotes the 'ground truth' of the HIT, i.e.

$$Pr(the\ ground\ truth\ of\ h_c\ is\ yes) = Pr(h_c)$$

while A_c denotes the 'crowdsourced answer' of the HIT, and

$$Pr(h_c\ is\ answered\ yes\ by\ a\ crowdsourcing\ worker) = Pr(A_c)$$

Because the crowd may make mistakes, they follow different distributions. Since the HITs are assumed to be answered independently, the relation between $Pr(h_c)$ and $Pr(A_c)$ can be expressed as the following equation.

$$Pr(A_c) = Pr(h_c) \cdot P_{cr} + (1 - Pr(h_c)) \cdot (1 - P_{cr}) \quad (6)$$

In Table 2, the values of $Pr(h_c)$ and $Pr(A_c)$ are demonstrated, which reveal the differences discussed above.

Then, we are interested in how this answer would affect the quality, i.e. how to compute $Q(T|A_c)$ and $Q(T|\neg A_c)$. In fact, the probability of each $t \in T$ is updated when the A_c is received. Intuitively, if t indicates contradicting content as A_c, then $Pr(t)$ should decrease; otherwise $Pr(t)$ should increase since it is further confirmed by the crowdsourcing worker. Explicitly, for each crowdsourced answer A_c, the probability of any $t \in T$ is modified as

$$\begin{aligned} Pr(t|A_c = yes) &= Pr(t) \cdot Pr(A_c = yes|t)/Pr(A_c) \\ Pr(t|A_c = no) &= Pr(t) \cdot Pr(A_c = no|t)/(1 - Pr(A_c)) \end{aligned} \quad (7)$$

where $Pr(A_c|t) = P_{cr}$ when $t \models c$ (i.e. the crowdsourcing worker confirms t), and $Pr(A_c|t) = 1 - P_{cr}$ when $t \models \neg c$ (i.e. the crowdsourcing worker disagrees with t). By recursively applying Equation 7, conflicting crowdsourced answers are integrated into the x-tuple t.

It is easy to perform the algebraic manipulations to show that, for any two answers A_{c_0} and A_{c_1}, we have

$$Pr(t|A_{c_0}, A_{c_1}) = Pr(t|A_{c_1}, A_{c_0}) \quad (8)$$

Equation 8 indicates that the final result of adjustment is independent of the sequence of the crowdsourced answers. In other words, when we have a deterministic set of HITs, it does not matter in what sequence the answers are used for adjusting the distribution in the x-tuple. In contrast, what matters is to determine the set of HITs to be asked, which is the core challenge addressed in Section 4.

3.2 Crowdsourcing Worker's Accuracy v.s. Data Quality

According to the Equations 6 and 7, we introduce two theorems to conclude the relation between the crowdsourcing worker's accuracy and the data quality.

Theorem 1 (Harmless Random). If a crowdsourcing worker randomly answers a HIT, i.e. $P_{cr} = 0.5$, then it does not affect the data quality.

PROOF. Please see appendix in the earlier version of this paper [7].

Now we compute the expectation of data quality after receiving the answer A_c, denoted by $\mathbb{E}Q(T|A_c)$.

Theorem 2 (Non-negative Expectation). Given an x-tuple T, if a HIT h_c is answered by a crowdsourcing worker with accuracy P_{cr}, the expected improvement of data quality of T is mathematically equivalent to the difference between 'the entropy of the crowdsourced answer of h_c' and the 'entropy of the crowdsourcing worker (Definition 5)', which is non-negative.

PROOF. Please see appendix in the earlier version of this paper [7].

From the above analysis, we always have non-negative expectation of quality improvement for asking each HIT. However, the quality of an x-tuple does not necessarily monotonically increase as receiving crowdsourced answers. It is possible for the quality to decrease, when a 'surprising' answer is received - a 'no' is answered by the crowdsourcing worker for a HIT with high probability to be correct, or vice versa. However, one can see that the data quality obviously converges to zero (i.e. the upper bound of data quality), since each HIT has a non-negative expectation on the change of quality. To achieve fast convergence, the key issue is how to select the HITs wisely.

4 K-HIT Selection

The core computational challenge we need to address is to select a collection of HITs with a cardinality constraint k. For the given budget B, we are interested in asking k HITs at each iteration, so there are totally $\lceil \frac{B}{k} \rceil$ iterations for budget to run out. When $k > 1$, different workers can then pick up these tasks and solve them in parallel, cutting down wall-clock time. Therefore, for a given budget of HITs B, larger k value leads to lower latency. However, we pay for this by having some questions answered that are not at the top of the list - we are issuing k good questions rather than only the very best one. At each iteration, we select k distinct HITs; but the same HIT may be repetitively selected in different iterations. Please note that the selection of HITs depends on the crowdsourced answers received from previous iterations. So the overall selection is an adaptive process, and how many times each HIT is asked during the whole process is determined by the crowdsourced answers and the proposed algorithms discussed in the rest of this section.

For a given x-tuple with m candidate distinct HITs, there are potentially C_m^k possible selections. In this section, we discuss how to select k HITs in order to maximize the expected quality improvement.

4.1 Deriving Objective Function

As stated, in our early version of the paper [7], we assumed that all crowdsourcing works have the same accuracy, which

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

is a very strong assumption. In this paper, we extend the model to allow diverse accuracy rates, and each worker cr_i has his/her own accuracy rate P_{cr_i}. Obviously the new model is more general and realistic. Surprisingly, after deleting the strong assumption of uniform accuracy rate, our algorithm is still optimal.

For a given set $S_h = \{h^i_{c_1}, h^i_{c_2}, \ldots, h^i_{c_k}\}$ of k HITs, each of which is assigned to one crowdsourcing work, we derive the expectation of quality improvement caused by the aggregation of crowdsourced answers of these HITs. Let A_{c_1}, A_{c_2},...,A_{c_k} denote the answers of the k HITs with accuracy P_{cr_1},P_{cr_2},...,P_{cr_k}, then A_{S_h} is a discrete random variable with 2^k possible outcomes, since each h_{c_i} is either answered 'yes' or 'no'. Let D_A and p_A be the domain and probability distribution of A_{S_h}, respectively, i.e.

$$\begin{aligned} &D_A = \{a_i | a_i \subseteq 2^{\{h_{c_1}, h_{c_2}, \ldots, h_{c_k}\}} \ and \\ &\forall h_{c_j} \in a_i \ , h_{c_j} \ is \ answered \ yes \ by \ crowd; \\ &\ and \ \forall h_{c_j} \notin a_i \ , h_{c_j} \ is \ answered \ no \ by \ crowd\} \\ &p_A = (Pr(a_1), Pr(a_2), ..., Pr(a_{2^k})) \end{aligned} \tag{9}$$

where $\sum_{i=1}^{2^k} Pr(a_k) = 1$. Thus the entropy of A_{s_h} is

$$H(A_{s_h}) = -\sum_{a_i \in D_A} Pr(a_i) \cdot \log Pr(a_i) \tag{10}$$

As we know, each c_i has a probability to show its ground truth, i.e. with P_{cr_i} to be true before crowdsourcing worker answer h_{c_n}. We view $U = \{c_n; n = 1, ..., k\}$ as a set of random variables which follow Bernoulli distribution and take value true/false. Let D_U and p_U be the domain and probability distribution of $\{c_n; n = 1, ..., k\}$ respectively. Each element of D_U is a sequence of true and false with a corresponding probability in p_U. Thus we have

$$\begin{aligned} &D_U = \{u_i | u_i = \{h^i_{c_1}, h^i_{c_2}, ..., h^i_{c_k}\}, h^i_{c_n} = ture \ or \ false\} \\ &p_U = (Pr(u_1), Pr(u_2), ..., Pr(u_{2^k})) \end{aligned} \tag{11}$$

where $|D_U| = 2^k$. we have

$$Pr(u_i) = \sum_{\substack{t_j \in T \\ n=1,\ldots,k \\ \forall c^i_n = ture \\ t \models c_i}} Pr(t_j) \tag{12}$$

We aim to find a set of k HITs such that their answers would lead to the maximal (expected) improvement of data quality. As shown in Theorem 4.1, under the assumption that crowdsourcing workers give correct answers with diverse accuracy probability, we prove that the uncertainty reduction by a set of HITs is equivalent to their joint entropy (denoted by H(A_{S_h})) minus sum of entropy values of crowdsourcing workers. So we investigate expected data quality by receiving the crowdsourced answer A_{S_h}. Given fixed k HITs with accuracy P_{cr_1}, P_{cr_2},..., P_{cr_k}, $H(A_{S_h}|T)$ is equivalent sum of entropy values of crowdsourcing workers, i.e. a constant. So, we have

$$\begin{aligned} \Delta Q_{S_h}(T) &= -\sum_{a_i \in D_A} Pr(a_i) \cdot \log Pr(a_i) - \sum_{i=1}^{k} H(cr_i) \\ &= H(A_{S_h}) - \sum_{i=1}^{k} H(cr_i) \end{aligned} \tag{13}$$

We highlight the conclusion of Equation 13 with the following theorem.

Theorem 3 (Objective Function). Given an x-tuple, a set of k cleaning HITs S_h and diverse accuracy rates of crowdsourcing workers, the expected quality improvement by asking these HITs is equivalent to difference between the 'joint entropy' and 'sum of entropy values of crowdsourcing workers'.

Since $\sum_{i=1}^{k} H(cr_i)$ is a constant, we only need to select HITs to maximize $H(A_{S_h})$. Formally, we have the following optimization goal:

$$S_h := \arg\max_{S_h} \ H(A_{S_h}) \tag{14}$$

4.2 Lower and Upper bounds

According to Theorem 3, we expect to select a set of k HITs, with highest $H(A_{S_h})$. For a given set HITs S_h, computing the exact value of $H(A_{S_h})$ encounters exponential complexity, due to the correlation among HITs and the crowd imperfection.

Note $H(A_{S_h}) = -\sum_{a_i \in D_A} Pr(a_i) \cdot \log Pr(a_i)$, the value of $Pr(a_i)$ should be computed first to further compute $H(A_{S_h})$. For k HITs, the answers in a_i are denoted by $A^i_{h_{c_1}}$, $A^i_{h_{c_2}}$,..., $A^i_{h_{c_k}}$. We present the formula for the computation of $Pr(a_i)$

$$Pr(a_i) = \sum_{j=1}^{2^k} Pr(u_j) \cdot Q_{ij} \tag{15}$$

where

$$\begin{aligned} Q_{ij} = &\prod_{\substack{n=1,..,k \\ c^j_n = ture, A_{c^i_n} = yes}} P_{cr_n} \cdot \prod_{\substack{n=1,..,k \\ c^j_n = ture, A_{c^i_n} = no}} (1 - P_{cr_n}) \\ &\cdot \prod_{\substack{n=1,..,k \\ c^j_n = false, A_{c^i_n} = yes}} P_{cr_n} \cdot \prod_{\substack{n=1,..,k \\ c^j_n = false, A_{c^i_n} = no}} (1 - P_{cr_n}) \end{aligned} \tag{16}$$

Clearly, the exact computation yields exponential complexity w.r.t k, which is inefficient when k is large. Thus, we are interested in reducing or avoiding the exact computation pertaining to $H(A_{S_h})$. By Equation 12 and Equation 15, we know that the number of elements with positive probability in D_U is less than or equal to $|T|$ and the time complexity of computing all $Pr(u_i)$ is bounded by $O(k|R|)$, but the time complexity of computing all $Pr(a_i)$ will be $O(2^k)$. So, we hope to bound $H(A_{S_h})$ by $H(S_h)$, in which $H(S_h)$ denotes the joint entropy of HITs in S_h.

In this subsection, we demonstrate how to bound $H(A_{S_h})$ by $H(S_h)$ through mathematical formula as follows. Meanwhile, we propose the upper and lower bounds that are tighter than the ones in [7]. The bounds can be computed in linear time and used to enable effective filtering (in Section 5.1) and heuristic estimation (in Section 5.2).

4.2.1 Upper Bound

Please note the difference between $H(S_h)$ and $H(A_{S_h})$ - $H(S_h)$ measures the randomness of the HITs, or more precisely, 'the ground truth of the HITs'; while $H(A_{S_h})$ measures the randomness of the 'crowdsourced answer of

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

the HITs'. Intuitively, this difference is caused by the fact that the crowdsourcing worker makes mistakes. Then by the chain rule of Shannon entropy, we have $H(S_h|A_{S_h}) = H(A_{S_h}, S_h) - H(A_{S_h})$, therefore

$$\begin{aligned} H(A_{S_h}) &= H(A_{S_h}, S_h) - H(S_h|A_{S_h}) \\ &= H(A_{S_h}|S_h) + H(S_h) - H(S_h|A_{S_h}) \end{aligned}$$

According to the conclusion in [7], we have an upper bound of $H(A_{S_h})$,

$$\begin{aligned} H(A_{S_h}) &= H(S_h) + \sum_{i=1}^{k} H(cr_i) - H(S_h|A_{S_h}) \\ &\leq H(S_h) + \sum_{i=1}^{k} H(cr_i) \end{aligned} \tag{17}$$

By Eq 10 and 15, we have

$$H(A_{S_h}) = \sum_{i=1}^{2^k}\sum_{j=1}^{2^k} Pr(u_j)Q_{ij} \log Pr(u_j)Q_{ij} \tag{18}$$

Note that $\sum_{j=1}^{2^k} Pr(u_j)$ = 1 and $P_{cr_n} \in$ [0.5,1], we have

$$\sum_{j=1}^{2^k} Pr(u_j)Q_{ij} \geq \min_j Q_{ij} = \prod_{n=1}^{k}(1 - P_{cr_n})$$

Therefore

$$\begin{aligned} H(A_{S_h}) &\leq -\sum_{i=1}^{2^k}\sum_{j=1}^{2^k} Pr(u_j)Q_{ij} \log \prod_{n=1}^{k}(1 - P_{cr_n}) \\ &= -\sum_{t=1}^{k} \log(1 - P_{cr_t}) \end{aligned} \tag{19}$$

Finally, by Equation 17 and Equation 19, we have the following upper bound

$$H(A_{S_h}) = \min\Big(H(S_h) + \sum_{i=1}^{k} H(cr_i), -\sum_{j=1}^{k} \log(1 - P_{cr_j})\Big) \tag{20}$$

4.2.2 Lower Bound

According to the conclusion in [7], we have

$$\begin{aligned} H(A_{S_h}) =& H(S_h) + \sum_{i=1}^{k} H(cr_i) - H(S_h|A_{S_h}) \\ \geq& H(S_h) + \sum_{i=1}^{k} H(cr_i) + \prod_{i=1}^{k} P_{cr_i} \log \prod_{i=1}^{k} P_{cr_i} \\ &+ (1 - \prod_{i=1}^{k} P_{cr_i}) \log(1 - \prod_{i=1}^{k} P_{cr_i}) \\ &- (1 - \prod_{i=1}^{k} P_{cr_i}) \log(2^k - 1) \end{aligned} \tag{21}$$

Similar with the upper bound, we have

$$\begin{aligned} H(A_{S_h}) &\geq -\sum_{i=1}^{2^k}\sum_{j=1}^{2^k} Pr(u_j) \cdot Q_{ij} \cdot \log \prod_{t=1}^{k} P_{cr_t} \\ &= -\sum_{i=1}^{k} \log P_{cr_i} \end{aligned} \tag{22}$$

Finally, by Equation 21 and Equation 22, we have the following lower bound

$$\begin{aligned} H(A_{S_h}) = \max\Big(& H(S_h) + \sum_{i=1}^{k} H(cr_i) \\ &+ \prod_{i=1}^{k} P_{cr_i} \cdot \log \prod_{i=1}^{k} P_{cr_i} - (1 - \prod_{i=1}^{k} P_{cr_i}) \cdot \log(2^k - 1) \\ &+ (1 - \prod_{i=1}^{k} P_{cr_i}) \cdot \log(1 - \prod_{i=1}^{k} P_{cr_i}), \\ &- \sum_{i=1}^{k} \log P_{cr_i}\Big) \end{aligned} \tag{23}$$

4.3 Computing the Bounds: X-partition Algorithm

From formula 17 and 22, one can see that we only need to compute the joint entropy $H(S_h)$, since the other components are all constant for a given x-tuple and a set of HITs. A naive way of computing the joint entropy is to compute all the marginal probabilities, hence yields to $O(2^k)$ complexity. However, with creating a small $O(|T|)$ in-memory index, we can achieve overall linear complexity - $O(k|T|)$.

We propose a novel algorithm named "X-partition". The intuition of the algorithm is, for each HIT h_c, the x-tuple T can be divided in to two parts, with $t_i \models c$ and $t_j \models \neg c$ respectively. As a consequence, k HITs can partition T into at most 2^k parts, and the aggregated probability of each part (i.e. the sum of probabilities of entries within a part) constitutes the marginal distribution of S_h. Note each part includes at least one tuple, so the total number of parts is at most $|T|$. Now we illustrate the algorithm in detail as following steps.

Step 1: Remove a HIT h_c from S_h.

Step 2: Partition the table into two parts T_0 and T_1, where $\forall t \in T_0, t \models c$, and $\forall t \in T_1, t \models \neg c$.

Step 3: Remove a HIT $h_{c'}$ from S_h

Step 4: For each of the current parts T_l, further divide it into two parts, T_{l0} and T_{l1}.

Step 5: repeat Step 3 and Step 4 until all k HITs are removed, then return $-\sum p \log p$, where p is the sum of probabilities of tuples within each part.

Correctness and Complexity: The correctness of the algorithm is straightforward. Each part T_l corresponds to a point of the marginal distribution of the S_h, so the entropy of these parts are equivalent to the joint entropy. In addition, the order of HITs would not affect the final partitioning result. At each iteration, for a given HIT, partitioning requires all the tuples, hence takes $O(|T|)$ time. As a result, the overall complexity becomes $O(k|T|)$, which is essentially linear w.r.t the size of input.

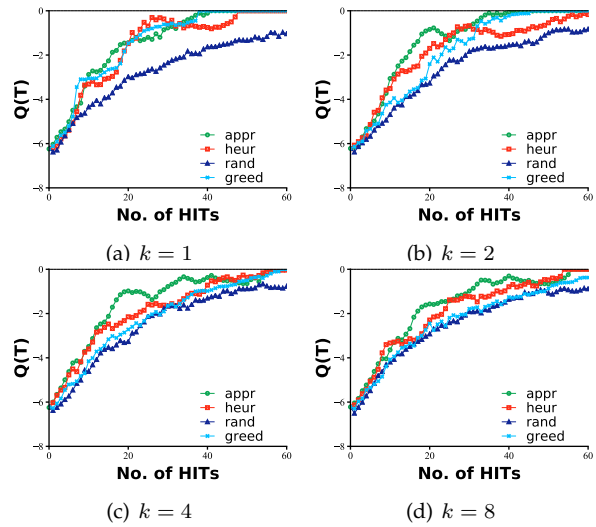
Running example : Given the example in Table 1 and 2, we consider a set of HITs $S_h = \{h_2, h_3\}$ are answered by two workers with P_{cr_2} = 0.8 and P_{cr_3} = 0.6. Then as shown in Table 2, we have - $Pr(h_2) = 0.5$,$Pr(h_3) = 0.4$, i.e. the probability of h_2 (h_3) has ground truth 'yes' are 0.5 (0.4); moreover, the probability of h_2 (h_3) being answered 'yes' by a crowdsourcing worker is 0.5 (0.45). The distributions of S_h and A_{S_h} is presented in the following table.

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

outcomes	$Pr(S_h)$	$Pr(A_{S_h})$
$h_2 \wedge h_3$	0	0.216
$h_2 \wedge \neg h_3$	0.5	0.284
$\neg h_2 \wedge h_3$	0.4	0.264
$\neg h_2 \wedge \neg h_3$	0.1	0.236

Note the distribution of S_h indicates the ground truth of the HITs, which is derived from the x-tuple in Table 1. Clearly, this distribution is irrelevant with the accuracy of the crowdsourcing worker. On the other hand, A_{S_h} denotes the distribution of the crowdsourced answer, which is computed according to formula 10. Having these two distributions, we can easily compute $H(S_h) = 1.36$, and $H(A_{S_h}) = 2.0$. We also have $\sum_{n=2}^{3} H(cr_n) = H(cr_2) + H(cr_3) = 0.72 + 0.97 = 1.69$ and -$\sum_{2}^{3} \log(1 - P(cr_n)) = 3.64$, then we have $H(S_h) + \sum_{n=2}^{3} H(cr_n) = 1.36 + 1.69 = 3.05 < 3.64$, by Eq 20, we can get the upper bound is $3.05 > 2.0$. Similarly, by the Eq 23, it is easy to get a lower bound is 1.23, and $1.23 < 2.0$.



(a) $k = 1$ (b) $k = 2$ (c) $k = 4$ (d) $k = 8$

Fig. 1. P_{cr_i} uniformly distributed in $[0.5, 1]$ and $k = 1, 2, 4, 8$

5 Hardness and Algorithms

One can see that the optimization problem derived in formula 14 is non-linear, due to the nature of Shannon entropy. In fact, the optimization problem is to find k HITs from the m candidate HITs, so the searching space is of complexity $O(m!)$. However, $H(A_{S_h})$ is the entropy of the crowdsourced answer of a set of HITs, and it is known that entropy is a submodular function [24]. Our optimization problem is essentially a sub-problem of the maximization of a general submodular function. We prove that finding the optimal solution for our optimization problem is NP-hard.

Theorem 4 (NP-hardness). It is NP-hard to find a set of k HITs such that the expected improvement of data quality is maximized.

PROOF. Please see appendix.

5.1 Approximation Algorithm

Although finding the optimal solution is intractable, it is known that the maximization of submodular functions can be approximated with a performance guarantee of $(1-1/e)$, by a greedy algorithm [25] - we iteratively select the best one HIT, given the ones selected so far.

Formally, we have the optimization function at the kth iteration:

$$x := \arg\max_{x} H(A_{(S_{k-1} \cup \{h_x\})}) \tag{24}$$

where S_{k-1} is the set of HITs selected from previous iterations.

At each iteration, we need to determine one HIT h_x out of m candidates, so $H(A_{(S_{k-1} \cup \{h_x\})})$ would be computed at most m times to find the maximum. Therefore, this approximation algorithm entails time complexity $O(2^k m)$, i.e. linear w.r.t the size of the x-tuple, but exponential w.r.t k. In order to further improve the efficiency, we propose two pruning methods, illustrated as follows.

Instance-level pruning:

We first adopt the lower upper bounds derived from Section 4.2. For a pair of HITs h_0 and h_1, if we have

$$H(A_{(S_{k-1} \cup \{h_0\})}).lb \geq H(A_{(S_{k-1} \cup \{h_1\})}).ub$$

then h_1 can be safely pruned. Please note that $A_{(S_{k-1} \cup \{h_0\})}.lb$ and $A_{(S_{k-1} \cup \{h_1\})}.ub$ denote the lower bound of $A_{(S_{k-1} \cup \{h_0\})}$ and the upper bound of $A_{(S_{k-1} \cup \{h_1\})}$, respectively. Both of them can be efficiently computed within $O(k)$ time by the X-partition algorithm in Section 4.3.

Algorithm-level pruning:

Submodularity can be exploited algorithmically to implement an accelerated variant of the greedy algorithm. In each of the k iterations, the greedy algorithm must identify the HIT with maximum marginal gain, given the HITs selected in the previous iterations. The key insight is that, as a consequence of submodularity of the objective function, the marginal benefit of any HIT is monotonically non-increasing during the iterations of the algorithm. In other words, for all h and k, the submodularity guarantees that $H(A_{(S_k \cup \{h\})}) \leq H(A_{(S_{k-1} \cup \{h\})})$. Therefore, we maintain a list of upper bounds (i.e.$A_{(S_{k-1} \cup \{h\})}$) on the marginal gains sorted in decreasing order. In each iteration, the algorithm extracts the HIT h_0 with the highest value from the ordered list. As a result, for all h_1, if we have

$$A_{(S_k \cup \{h_0\})} \geq A_{(S_{k-1} \cup \{h_1\})} \tag{25}$$

then, h_1 can be safely pruned.

5.2 Heuristic Algorithm

In the approximation algorithm above, after applying the pruning methods, we may still have to compute the exact value of $H(A_{(S_{k-1} \cup \{h_x\})})$ for several times, each of which entails exponential complexity w.r.t k. This can be inefficient when k is large. In this subsection, we use the average of upper bound and lower bound to estimate the value of $H(A_{(S_{k-1} \cup \{h_x\})})$ for HIT selection. Explicitly, we apply the following estimator:

$$H(A_{(S_{k-1} \cup \{h_x\})}) \approx \frac{A_{(S_{k-1} \cup \{h_x\})}.lb + A_{(S_{k-1} \cup \{h_x\})}.ub}{2} \tag{26}$$

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

We compute the estimator with formula 26, and select the HIT with the highest estimated value. Therefore, considering the entire heuristic algorithm, we do not need to compute any exact value of $H(A_{(S_{k-1}\cup\{h_x\})})$. In the experiments, we show that the effectiveness of this heuristic algorithm is comparable with the approximation algorithm.

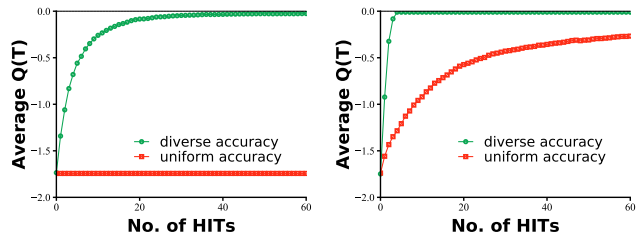
The heuristic algorithm is particular useful when k is large - it entails very short (linear w.r.t k) execution time for the program to select HITs. On the other hand, recall that larger k would lead to less overall time cost of crowdsourcing (i.e. the time waiting for the crowd to return answers), since multiple workers can take HITs in parallel. So the heuristic algorithm is recommended over the approximation algorithm when the user expects short overall running time (program execution time + crowdsourcing). In other words, when the budget is the main constraint, small k and the approximation algorithm should be adopted, whereas the heuristic algorithm with a large k value is suggested if the time-efficiency is the primary constraint.

6 Experimental Evaluation

The goal of our experiments contains following aspects, 1) we study the effect of different parameters for quality improvement with respect to our approximation algorithm and heuristic algorithm, especially the effect of considering different accuracy rates of the crowd; 2) we compare the two proposed algorithms with baseline algorithms in term of hardness reduction; 3) we compare our crowdsourcing based algorithm with automatic cleaning tools such as HoloClean, and discuss the combination of the two approaches.

The rest of this section includes following content.

- Firstly, we present in Section 6.1 the necessity of considering different accuracy rates. As in our previous conference paper [7], the workers in the crowd are considered to have the accuracy rate, the performance is obviously worse compared with our method in this paper.
- Secondly, in Section 6.2, we compare our approximation algorithm and heuristic algorithm with two baselines, with different numbers of selected HITs per iteration. The first baseline just selects HITs randomly, and the other one is a greedy strategy which commits to the largest reduction of uncertainty each iteration.
- Thirdly, in Section 6.3, we approximate the algorithm facilitated by the instance-level pruning and algorithm-level pruning, and then discuss the efficiency.
- Fourthly, we test our methods in Section 6.4 with Amazon Mechanical Turk, which is a well-known public crowdsourcing platform. Due to page limits, we omit corresponding experiment results with the uniform accuracy rate in our previous conference paper.
- Finally, we compare our crowdsourcing based algorithm with automatic cleaning tools such as HoloClean [26], and discuss the combination of the two approaches in Section 6.5.



(a) $P^l_{cr} = 0.1$, $P^h_{cr} = 0.9$ (b) $P^l_{cr} = 0.5$, $P^h_{cr} = 1.0$

Fig. 2. Diverse accuracy vs. uniform accuracy

6.1 Diverse Accuracy vs. Uniform Accuracy

We first present the necessity of introducing diverse accuracy rates into the framework. In our previous conference paper [7], the workers in the crowd are considered to have accuracy rates. Such an assumption is acceptable when the crowdsourcing workers in reality have very similar quality in term of answer accuracy. However, when there is obvious accuracy diversity in the crowd, which is very common in real applications, the selection of the best HITs will be inaccurate due to the insufficient estimation of the hardness reduction.

Consider such a crowd of workers, in which a half of the population have quite low accuracy rates, say around P^l_{cr}, and the other half has high accuracy rates, around P^h_{cr}. Now we mixed up these two subgroup of crowdsourcing workers and test the performance on a real dataset *Hospital* in [26], which contains 1000 x-tuples and 5000 tuples. As the algorithm in [7] can not distinguish workers with different accuracy rates, we use the average accuracy rate of the crowd as input. Our algorithm with diverse accuracy rates shows much better performance. With ($P^l_{cr} = 0.1, P^h_{cr} = 0.9$) and ($P^l_{cr} = 0.5, P^h_{cr} = 1.0$), the performance of our method is significantly better than the algorithm that assumes the same accuracy rate for all users. Particularly in Figure 2(a), with $P^l_{cr} = 0.1$ and $P^h_{cr} = 0.9$, the previous version of the algorithm in [7] can barely handle the problem. Under the budget limit, the uncertainty of the data is not reduced at all, which is consistent with Theorem 1.

6.2 Performance comparison on Synthetic Data

In this subsection, we experiment on synthetic data with a simulated crowd. We generate $10,000$ x-tuples in total, each of which contains 50 tuples and 20 attributes, and report the average performance. For each published HIT, we randomly generate an accuracy rate $P_{cr} \in [0.5, 1]$ with uniform distribution. Third, given a HIT, we generate the correct yes-no answer with probability P_{cr} (i.e. generate the wrong answer with probability 1 - P_{cr}, and then return the answer and P_{cr} as the inputs for adjustment (Section 3.1). Since each x-tuple usually contains a small number of tuples (less than 15 in the real datasets used in Section 6.4), the synthetic data is fairly large enough to test on the scalability of the proposed algorithms. We set the total budget of HITs $B = 60$ for a given x-tuple, and at each iteration we select k HITs, so there are totally $\lceil 60/k \rceil$ iterations (there are maybe less than k HITs in the last iteration). Then we demonstrate the performances of the approximation algorithm (***appr***),

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

the heuristic algorithm (***heur***), the greedy algorithm (***greed***) and a naive algorithm (***rand***), which selects HITs randomly. Please note that the proposed algorithms are both **linear** w.r.t the size of x-tuples. The main computational challenge is that the time cost of *appr* is exponential w.r.t k. It'll be thoroughly evaluated in the experiments.

We first present the effectiveness of the algorithm with varying k, which is the number of questions selected for each iteration. For a given budget $B = 60$, larger k leads to less iterations. With more than one HITs on the crowd ($k > 1$), different workers can take them on parallel. We set k to $1, 2, 4, 8$, and test the performances of the algorithms above. As shown in Figure 1, smaller k tends to be more effective on quality improvement. In fact, the larger k is, the less advantage *appr* and *heur* have comparing to a random selection or a greedy one. This is because each HIT is selected based on crowdsourced answers of HITs selected in previous iterations. Recall that we select k out of m candidates at each iteration, so when $k = m$, *appr* and *heur* are the same as random selection, i.e. select all of the HITs we have.

We now present the performances of the approximation algorithm (*appr*), the heuristic algorithm (*heur*), the greedy algorithm (*greed*) and a naive algorithm (*rand*). As indicated in Figure 1, *heur*, *appr* and *greed* significantly outperform *rand* in all cases. It's worth mentioning that *appr*, *heur* and *greed* have almost the same performance when $k = 1$, because at each iteration, the three algorithms would always select the HIT with probability closest to 0.5 to achieve their respective local optimal. When $k > 1$, *heur* and *appr* notably outperform than *greed*, since the greedy algorithm is always looking for the local optimal solution. It is difficult to greedily get the global optimal solution when multiple HITs are given at the same time. We can also observe that *heur* and *appr* occasionally perform similarly; whereas *appr* performs slightly better than *heur* and *greed* in most cases in terms of the number of HITs.

6.3 Approximation and Efficiency

In Section 5.1 and 5.2, the approximation algorithm (*appr*) and the heuristic algorithm (*heur*) are proposed as solutions for k-HIT selection. Recall that *appr* generates near-optimal solutions with approximation guarantee, but it encounters high computational cost when k is large. Therefore, two filtering techniques are proposed to improve the efficiency. In the following, we demonstrate the computational cost of the proposed algorithms in Figure 3. In particular, the curves labeled with *appr_i_pruning* and *appr_a_pruning* represent the approximation algorithm facilitated by the instance-level pruning and algorithm-level pruning, respectively. In addition, the curve *appr_both_prunings* is the result with the both pruning methods used. The pruning techniques turn out to significantly improve the efficiency of the algorithm *appr*, and the time cost of *heur* is linear w.r.t k. We can observe that the instance-level pruning is more effective than the algorithm-level one. Moreover, combining the pruning methods is better than any single one of them. We also run the exact solution of k-HIT selection, which enumerates all the possible size-k sets of HITs. However, due to the NP-hardness of the exact solution, we cut off the experiment

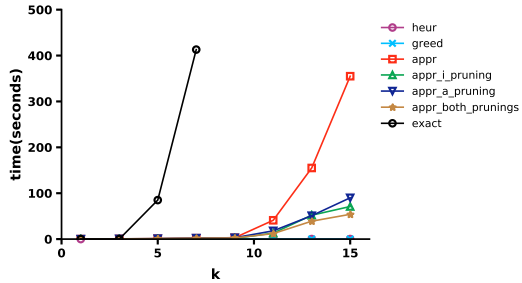


Fig. 3. Efficiency of k-HIT Selection Algorithms (new upper and lower bounds)

when $k > 7$, which can not terminate in acceptable time. As mentioned previously, we derive new lower and upper bounds for effective filtering and estimation in this paper. Both of the new bounds are tighter than the ones in our previous conference paper [7], and the tighter bounds benefit on the efficiency of all algorithms, compared with Figure 2 in [7].

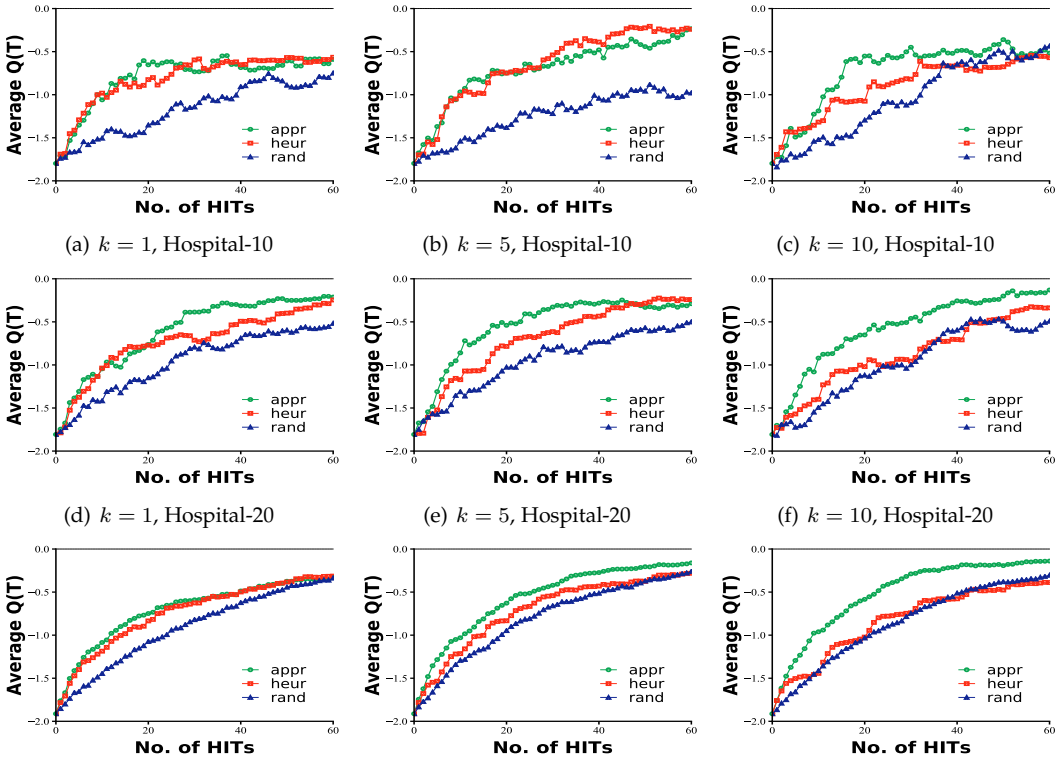
6.4 Performance Testing on Amazon Mechanical Turk

We implement our two approaches on Amazon Mechanical Turk (AMT), which is a widely used crowdsourcing marketplace. We tested a relatively large dataset from the real-world raw data *Hospital* [26]. The tested data from *Hospital* includes 100 x-tuples, each of which has 19 attributes and around 5 tuples. Thus there are in total more than 1250 tuples and 6000 HITs. For each x-tuple in the dataset, we use the label in the raw data as the ground truth. Each worker is required to take an unpaid *Qualification Test* containing 10 sample questions. A worker is accepted only if he/she correctly answers at least 6 questions in the qualification test. Each worker is only allowed to answer one HIT from each x-tuple, but they may take multiple HITs from different x-tuples. In order to estimate the crowd's accuracies, we first publish 30 testing HITs for each dataset. Then we estimate the crowd's accuracies with these testing HITs as well as the questions answered in the qualification tests. After the 30 testing HITs being finished, the crowd's accuracies are between 0.64 to 0.86. We randomly sample a precision between these two values to set up the worker's accuracy rate for each crowdsourcing round.

We set the budget $B = 60$, and each HIT is awarded 0.04 USD. We compare *appr*, *heur* and *rand* with $k = 1, 5, 10$. Please note a budget is set up for each x-tuple, so users are allowed to set different budgets for different data instances. For the scalability concerning the number of x-tuples, if the same budget is assumed to be set for all x-tuples, the total number of HITs is simply equal to the 'number of x-tuples' multiplies the 'budget', i.e. linear w.r.t the number of x-tuples. We test the competing algorithms with three groups of experiments, respectively on 10, 20 and 100 x-tuples with k=1, 5 and 10, and then plot the average Q(T). The average performances are demonstrated in Figure 4. In terms of the improvement of data quality, one can see that the performance is basically consistent with the simulation - *appr* and *heur* outperform *rand* in all cases in terms of the number of HITs. Moreover, with larger amount of x-tuples,

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering



(a) $k = 1$, Hospital-10 (b) $k = 5$, Hospital-10 (c) $k = 10$, Hospital-10

(d) $k = 1$, Hospital-20 (e) $k = 5$, Hospital-20 (f) $k = 10$, Hospital-20

(g) $k = 1$, Hospital-100 (h) $k = 5$, Hospital-100 (i) $k = 10$, Hospital-100

Fig. 4. Testing on Amazon Mechanical Turk

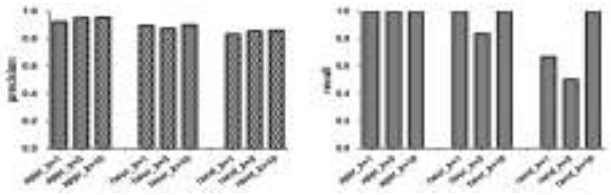


Fig. 5. Data Quality with Budget Constraint - Precision and Recall

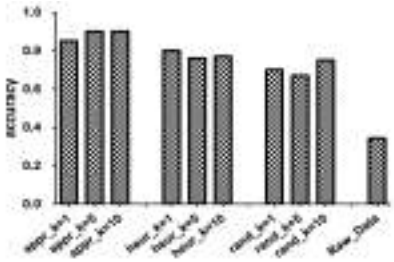


Fig. 6. Data Accuracy Improvement

the updated data quality shows smoother improvement during crowdsourcing. In the comparison with HoloClean in Section 6.5, We include the dataset *Hospital* and another large dataset *Flights*, and set the accuracy given to the algorithms to be a random value between 0.64 and 0.86 (lower and upper bound from the qualification tests for the crowd). We will show that, with the very large amount of HITs, the average Q(T) is almost monotonic during crowdsourcing. We conclude that for the two large real-world datasets, our algorithms perform quite well.

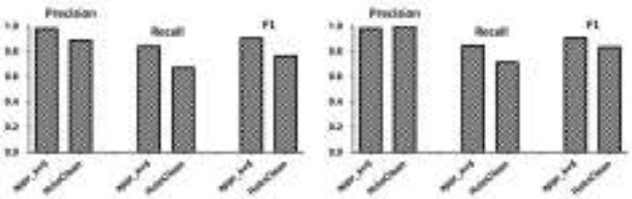
Lastly, we verify the correctness of our approaches, by evaluating the accuracy, precision and recall of the best tuple in each x-tuple, i.e. the tuple with the highest possibility after cleaning. The accuracy is the ratio between the number of best tuples that are the same as the ground truth and the total number of x-tuples. Precision is computed as the ratio of the correctly reported correspondences out of all reported correspondences. Recall is computed as the ratio of correctly reported correspondences out of all correspondences in the ground truth. The data accuracy improvement is illustrated in Figure 6. Please note that "Raw_Data" denotes the data accuracy before cleaning. One can see that the data accuracy is significantly improved after cleaning. Moreover, for *appr* and *heur*, the lower value of k tend to have higher accuracy. This is because smaller k indicates better HIT selections, but longer latency. Figure 5 illustrates the quality of the best tuples after uncertainty reduction with budget B = 60. From the experimental results, we conclude that *appr* and *heur* significantly outperform than *rand* in all cases, *appr* and *heur* occasionally perform similarly. This is basically consistent with the experimental results in Figure 6.

6.5 Comparison and Combination with Automatic Cleaning Method

We compare our crowdsourcing based approach with some automatic data cleaning methods such as HoloClean [26]. HoloClean is a framework for holistic data repairing driven by probabilistic inference. It to repair dirty data by using some explicit rules or constraints from human experts, some

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering



(a) *Flights* (b) *Hospital*

Fig. 7. Comparison with HoloClean on *Flights* and *Hospital*

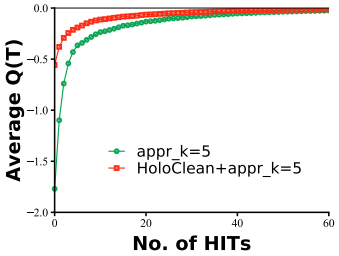


Fig. 8. Combining crowd-aided algorithm with HoloClean on *Hospital*

additional external information and quantitative statistics. Compared to our crowd-aided method, the human expertise is embedded in the explicit rules or constraints, instead of the crowdsourced answers. The repair of the data by HoloClean consists of two steps, namely noisy cell recognition and data correction. In the former step, the system finds all possible noisy cell by either some predefined first-order logical rules, or some external data information, or some detection techniques for data abnormality. Then in the second step, HoloClean proposes probability inference model and repair the data in the noisy cells, under the weak supervision by the above detection results. After repairing, the system outputs the distribution of possible values of each noisy cells.

To compare our crowd-aided algorithm with the automatic cleaning approach, we formulate the datasets used in [26] as x-tuples. Each item in the raw data is regarded as an x-tuple, which is split into several candidate tuples, according to the candidate values of the attributes, and the distribution of the tuples is initialized with the statistical distribution. These x-tuples are then given to approximation algorithm to get a updated distribution of the candidate tuples for each x-tuple. Finally, with the updated distributions of the candidate tuples (which are joint probability distributions), we compute the marginal probability distributions on the candidate values of the noisy cells. We use the labels in dataset as the ground truth to evaluate the output. Thus, we can compare the methods on two large real datasets, *Flights* and *Hospital*, in term of the precision, the recall, and F1 score. The dataset *Flights* contains 57,246 x-tuples and 286,230 tuples, and *Hospital* contains 1000 x-tuples and 5000 tuples. Note that the performance of HoloClean is effected by a pruning threshold τ, which cuts off some minor noises and allows users to tradeoff the scalability and the quality of repairs obtained by it. Higher values of τ lead to smaller domains for random variables, thus, smaller size factor graphs and more efficient inference. For both of the datasets, we compare our method with the best performance of HoloClean, namely *Flights* with $\tau = 0.3$ and *Hospital* with $\tau = 0.5$. As shown in Figure 7, our method performs better than HoloClean on both datasets.

It worth mentioning that the crowd-aided method and the automatic cleaning can be combined. The output of HoloClean can be used to initialize the distributions of the candidate tuples before crowdsourcing. We compare the performances of the approximation algorithm with two different initial distributions, one of which uses the joint distributions computed according to the output of HoloClean, and the other one uses the joint distributions computed according to statistical distributions in the raw data. The average $Q(T)$ of 1000 x-tuples from *Hospital* is plotted in Figure 8. The result shows that 1) the output of HoloClean is further improved by crowdsourced answers, and 2) the distributions from HoloClean as initial input can also accelerate the convergence of our algorithm.

7 RELATED WORKS

7.1 Cleaning Uncertain Data

Data cleaning techniques have been developed for decades by the database community [27], [28]. In particular, there is a number of works studying the issues of cleaning uncertain data [5], [29]. Given that a limited amount of resources to clean the database, [4] describes a technique for choosing the set of uncertain objects to be cleaned, in order to achieve the best improvement in the quality of query answers. They develop a quality metric, namely PWS-quality, for a probabilistic database, and they investigate how such a metric can be used for data cleaning purposes. Please note PWS-quality is essentially the negative value of Shannon entropy, which is the same metric we adopt in this paper. In [5], the authors extend the approach of [4] to support top-k queries. [29] provides an analysis on the sensitivity of a probabilistic query answer to the input data. [30] applies user's feedback to improve the quality of an uncertain database integrated from different sources.

This paper distinguishes itself from the existing works in two perspectives as follows. First, we consider the crowdsourcing workers as a special resource for cleaning uncertain data. The speciality includes 1) the crowdsourcing workers works best when the tasks are broken down into small pieces; and 2) crowdsourcing workers may provide wrong information. Second, we focus on cleaning individual data instances (i.e. x-tuples), not query answers. By improving the quality of data instances, any related queries can be benefited.

7.2 Crowdsourcing

The recent development of crowdsourcing brings us a new opportunity to engage human intelligence into the process of answering queries [31], [32], [33]. Crowdsourcing provides a new problem-solving paradigm [34], [35], which has been blended into several research communities, including database and data mining [36]. In the practical viewpoint, [37] proposed and develop a query processing system using microtask-based crowdsourcing to answer queries. In [38], crowdsourcing is used for top-k query processing over

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

uncertain data and [39] studied knowledge base semantic integration using crowdsourcing. Moreover, in [40], a declarative query model is proposed to cooperate with standard relational database operators. In addition, in the viewpoint of theoretical study, many fundamental queries have been extensively studied, including filtering [22], max [19], sorting [15], join [16], etc. Besides, crowdsourcing-based solutions of many complex algorithms are developed, such as categorization based on graph search [13], clustering [41], entity resolution [17], [18], tagging in social networks [42], trip planning [43], topic discovery [44] etc. In the recent work [45], the authors use crowdsourcing techniques to improve the results of information extraction systems. Our work is essentially different from [45] discussed as follows. First, during the selection of HITs, [45] uses the 'token entropy' and 'mutual information' to select HITs, but does not consider the error rate of the crowd, which is one of the core challenges addressed in our model. Second, [45] only focuses on data from information extraction systems, and their methods cannot be used when the uncertainty is caused by other reasons.

8 Conclusion

In this paper, we propose utilizing the power of crowdsourcing on cleaning uncertain data. In many circumstances, the uncertainty is caused by the fact that machines cannot fully capture the human-intuitive semantics (e.g. images, human languages). Therefore, we are motivated to use crowdsourcing to eliminate the uncertainty of the data. In particular, we first prove that the crowdsourcing workers, although noisy, is a profitable approach for cleaning uncertain data. Then, we optimize the approach by efficiently identifying the most valuable set of questions (HITs).

Uncertainty is inherited in many modern applications, such as information extraction, entity resolution, schema matching, and truth discovery. We believe that adopting crowdsourcing as a new component of a DBMS would be extremely conductive for the improving the quality of uncertain data, hence effectively improve the overall performance. Our work represents an initial solution towards cleaning uncertain data with crowdsourcing.

References

[1] R. Gupta and S. Sarawagi, "Creating probabilistic databases from information extraction models," in *VLDB*, 2006, pp. 965–976.

[2] P. Singla and P. Domingos, "Entity resolution with markov logic," in *ICDM*, 2006, pp. 572–582.

[3] A. Gal, "Managing uncertainty in schema matching with top-k schema mappings," pp. 90–114, 2006.

[4] R. Cheng, J. Chen, and X. Xie, "Cleaning uncertain data with quality guarantees," *PVLDB*, vol. 1, no. 1, pp. 722–735, 2008.

[5] L. Mo, R. Cheng, X. Li, D. W. Cheung, and X. S. Yang, "Cleaning uncertain data for top-k queries," in *ICDE*, 2013, pp. 134–145.

[6] X. Yu, G. Li, Y. Zheng, Y. Huang, S. Zhang, and F. Chen, "Crowdota: An online task assignment system in crowdsourcing," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1629–1632.

[7] C. J. Zhang, L. Chen, Y. Tong, and Z. Liu, "Cleaning uncertain data with a noisy crowd," in *31st IEEE International Conference on Data Engineering, ICDE 2015, Seoul, South Korea, April 13-17, 2015*, 2015, pp. 6–17.

[8] P. Agrawal, O. Benjelloun, A. D. Sarma, C. Hayworth, S. U. Nabar, T. Sugihara, and J. Widom, "Trio: A system for data, uncertainty, and lineage," in *VLDB*, 2006, pp. 1151–1154.

[9] Y. Tong, L. Chen, and B. Ding, "Discovering threshold-based frequent closed itemsets over probabilistic data," in *IEEE 28th International Conference on Data Engineering (ICDE 2012), Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012*, 2012, pp. 270–281.

[10] Y. Tong, L. Chen, Y. Cheng, and P. S. Yu, "Mining frequent itemsets over uncertain databases," *PVLDB*, vol. 5, no. 11, pp. 1650–1661, 2012.

[11] R. Cheng, "Querying and cleaning uncertain data," in *Quality of Context*, ser. Lecture Notes in Computer Science, K. Rothermel, D. Fritsch, W. Blochinger, and F. Dürr, Eds. Springer Berlin Heidelberg, 2009, vol. 5786, pp. 41–52. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-04559-2_4

[12] H. Park and J. Widom, "Query optimization over crowdsourced data," *PVLDB*, vol. 6, no. 10, pp. 781–792, 2013.

[13] A. G. Parameswaran, A. D. Sarma, H. Garcia-Molina, N. Polyzotis, and J. Widom, "Human-assisted graph search: it's okay to ask questions," *PVLDB*, vol. 4, no. 5, pp. 267–278, 2011.

[14] A. Kittur, B. Smus, S. Khamkar, and R. E. Kraut, "Crowdforge: Crowdsourcing complex work," in *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '11. New York, NY, USA: ACM, 2011, pp. 43–52.

[15] A. Marcus, E. Wu, D. R. Karger, S. Madden, and R. C. Miller, "Human-powered sorts and joins," *PVLDB*, vol. 5, no. 1, pp. 13–24, 2011.

[16] J. Wang, G. Li, T. Kraska, M. J. Franklin, and J. Feng, "Leveraging transitive relations for crowdsourced joins," in *SIGMOD Conference*, 2013, pp. 229–240.

[17] J. Wang, T. Kraska, M. J. Franklin, and J. Feng, "Crowder: Crowdsourcing entity resolution," *PVLDB*, vol. 5, no. 11, pp. 1483–1494, 2012.

[18] S. E. Whang, P. Lofgren, and H. Garcia-Molina, "Question selection for crowd entity resolution," *PVLDB*, vol. 6, no. 6, pp. 349–360, 2013.

[19] S. Guo, A. G. Parameswaran, and H. Garcia-Molina, "So who won?: dynamic max discovery with the crowd," in *SIGMOD Conference*, 2012, pp. 385–396.

[20] S. B. Davidson, S. Khanna, T. Milo, and S. Roy, "Using the crowd for top-k and group-by queries," in *ICDT*, 2013, pp. 225–236.

[21] A. D. Sarma, A. Parameswaran, H. Garcia-Molina, and A. Halevy, "Finding with the crowd," Stanford University, Technical Report. [Online]. Available: http://ilpubs.stanford.edu:8090/1049/

[22] A. G. Parameswaran, H. Garcia-Molina, H. Park, N. Polyzotis, A. Ramesh, and J. Widom, "Crowdscreen: algorithms for filtering data with humans," in *SIGMOD Conference*, 2012, pp. 361–372.

[23] X. Liu, M. Lu, B. C. Ooi, Y. Shen, S. Wu, and M. Zhang, "Cdas: A crowdsourcing data analytics system," *PVLDB*, vol. 5, no. 10, pp. 1040–1051, 2012.

[24] C. G. Andreas Krause, "A note on the budgeted maximization of submodular functions," School of Computer Science, Carnegie Mellon University, Tech. Rep., March 2005.

[25] S. Khuller, A. Moss, and J. Naor, "The budgeted maximum coverage problem," *Inf. Process. Lett.*, vol. 70, no. 1, pp. 39–45, 1999.

[26] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "repairs with probabilistic inference," *Proc. VLDB Endow.*, vol. 10, no. 11, pp. 1190–1201, 2017.

[27] V. Ganti, *Data Cleaning*, ser. Synthesis Lectures on Data Management. Morgan & Claypool Publishers, 2011. [Online]. Available: http://books.google.com.hk/books?id=V12tXwAACAAJ

[28] E. Rahm and H. H. Do, "Data cleaning: Problems and current approaches," *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.

[29] B. Kanagal, J. Li, and A. Deshpande, "Sensitivity analysis and explanations for robust query evaluation in probabilistic databases," in *SIGMOD Conference*, 2011, pp. 841–852.

[30] M. van Keulen and A. de Keijzer, "Qualitative effects of knowledge rules and user feedback in probabilistic data integration," *VLDB J.*, vol. 18, no. 5, pp. 1191–1217, 2009.

[31] A. Doan, R. Ramakrishnan, and A. Y. Halevy, "Crowdsourcing systems on the world-wide web," *Commun. ACM*, vol. 54, no. 4, pp. 86–96, 2011.

[32] G. Li, J. Wang, Y. Zheng, and M. J. Franklin, "Crowdsourced data management: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 9, pp. 2296–2319, 2016.

[33] A. I. Chittilappilly, L. Chen, and S. Amer-Yahia, "A survey of general-purpose crowdsourcing techniques," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 9, pp. 2246–2266, 2016.

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2020.3027545, IEEE Transactions on Knowledge and Data Engineering

[34] D. C. Brabham, "Crowdsourcing as a model for problem solving an introduction and cases," *Convergence February 2008 vol. 14 no. 1 75-90*, 2008.

[35] T. Malone, R. Laubacher, and C. Dellarocas, "Harnessing crowds: Mapping the genome of collective intelligence," MIT, Sloan School of Management, Massachusetts Institute of Technology, Cambridge, MA, USA, Research Paper No. 4732-09, February 2009, sloan Research Paper No. 4732-09.

[36] Y. Zheng, G. Li, Y. Li, C. Shan, and R. Cheng, "Truth inference in crowdsourcing: Is the problem solved?" *Proceedings of the VLDB Endowment*, vol. 10, no. 5, pp. 541–552, 2017.

[37] A. Feng, M. J. Franklin, D. Kossmann, T. Kraska, S. Madden, S. Ramesh, A. Wang, and R. Xin, "Crowddb: Query processing with the vldb crowd," *PVLDB*, vol. 4, no. 12, pp. 1387–1390, 2011.

[38] E. Ciceri, P. Fraternali, D. Martinenghi, and M. Tagliasacchi, "Crowdsourcing for top-k query processing over uncertain data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 1, pp. 41–53, 2015.

[39] R. Meng, L. Chen, Y. Tong, and C. Zhang, "Knowledge base semantic integration using crowdsourcing," *IEEE transactions on knowledge and data engineering*, vol. 29, no. 5, pp. 1087–1100, 2017.

[40] A. G. Parameswaran and N. Polyzotis, "Answering queries using humans, algorithms and databases," in *CIDR*, 2011, pp. 160–166.

[41] R. Gomes, P. Welinder, A. Krause, and P. Perona, "Crowdclustering," in *NIPS*, 2011, pp. 558–566.

[42] M. Das, S. Thirumuruganathan, S. Amer-Yahia, G. Das, and C. Yu, "Who tags what? an analysis framework," *PVLDB*, vol. 5, no. 11, pp. 1567–1578, 2012.

[43] H. Kaplan, I. Lotosh, T. Milo, and S. Novgorodov, "Answering planning queries with the crowd," *PVLDB*, vol. 6, no. 9, pp. 697–708, 2013.

[44] Y. Tong, C. C. Cao, and L. Chen, "Tcs: Efficient topic discovery over crowd-oriented service data," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '14. New York, NY, USA: ACM, 2014, pp. 861–870.

[45] S. Goldberg, D. Z. Wang, and T. Kraska, "Castle: Crowd-assisted system for textual labeling & extraction," in *HCOMP*, 2013.

Chen Zhang received the PhD degree from the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, in 2015. He is currently a postdoctoral research fellow with the Hong Kong University of Science and Technology as well as an associate professor at the Shandong University of Finance and Economics. His research interests include crowdsourcing and data integration. He is a member of the IEEE.

Weiteng Xie received the Bachelor degree from the College of Physics and Electronic Science, Hubei Normal University, in 2017. He is currently a Master student in the College of Computer Science and Software Engineering, Shenzhen University and Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen University, China.

Nan Liu is currently an undergraduate student in the College of Engineering Computer Science Engineering Division from University of Michigan, Ann Arbor, USA, and is currently visiting Shenzhen University. His research interests include crowdsourcing, crowd-aided uncertainty reduction, path selection, etc.

Qifan Li received the Bachelor degree from the College of Mechanical Engineering, Dongguan University of Technology, in 2018. He is currently a Master student in the College of Computer Science and Software Engineering, Shenzhen University and Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen University, China.

Kaishun Wu received the PhD degree in Department of Computer Science and Engineering, Hong Kong University of Science and Technology, in 2011. He is currently distinguished professor in the College of Computer Science and Software Engineering, Shenzhen University and Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen University, China. He is a member of the IEEE.

Jiang Di received the PhD degree in computer science from the Hong Kong University of Science and Technology in 2014. He is currently a senior scientist at WeBank AI. His research interests include information retrieval, natural language processing and massive data management.

Peiguang Lin received the the PhD degree in the School of Computer Science and Technology from Beijing Institute of Technology, Beijing, China, in 2006. Dr. Lin is an associate professor at the School of Computer Science and Technology, Shandong University of Finance and Economics, Shandong, China. His current research interests include massive data processing and integrated, and network information processing.

Lei Chen received the PhD degree in computer science from the University of Waterloo, Canada, in 2005. He is currently a professor in the Department of Computer Science and Engineering, Hong Kong University of Science and Technology. His research interests include crowdsourcing over social media, social media analysis, probabilistic and uncertain databases, and privacy-preserved data publishing. He is a member of the IEEE.

Haodi Zhang received the PhD degree from the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, in 2016. He is currently an assistant professor in the College of Computer Science and Software Engineering, Shenzhen University and Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen University, China.

1041-4347 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on July 09,2021 at 07:21:32 UTC from IEEE Xplore. Restrictions apply.